

6a - Programmazione MPI – mpirun e esercizi di base

mpirun e mpicc sono gli strumenti principali per l'utilizzo del modello a memoria distribuita con MPI nei sistemi HPC.

La prima parte del laboratorio, riguardo il funzionamento di mpirun, mpicc e gli esercizi di base, può essere svolta sul PC personale o sul cluster HPC.

La seconda parte del laboratorio riguarda l'utilizzo di mpirun via Slurm, da eseguire sul cluster.

Ambiente MPI nel container ubuntu-hpc

openMPI 4.1.6 (vedi mpirun -version) - gcc 13.3.0

Ambiente MPI sul cluster HPC

Sul cluster HPC sono installate diverse implementazioni di MPI-1 MPI-3 MPI-4 in openMPI e Intel-MPI, attivabili via **module load**:

MPI	Compiler	module load
openmpi (1.10.7)	gcc5 (5.4.0)	gnu openmpi
openmpi3 (3.1.4)	gcc7 (7.3.0)	gnu7 openmpi3
openmpi3 (3.1.6)	gcc8 (8.3.0)	gnu8 openmpi3
openmpi4 (4.1.1)	gcc8 (8.3.0)	gnu8 openmpi4
intelmpi3.1 (2019.5.281)	intel-compiler (2019.5)	intel impi
openmpi3 (3.1.4)	intel-compiler (2019.5)	intel openmpi3

mpirun

mpirun è il comando che si occupa della creazione e dell'esecuzione dei processi MPI.

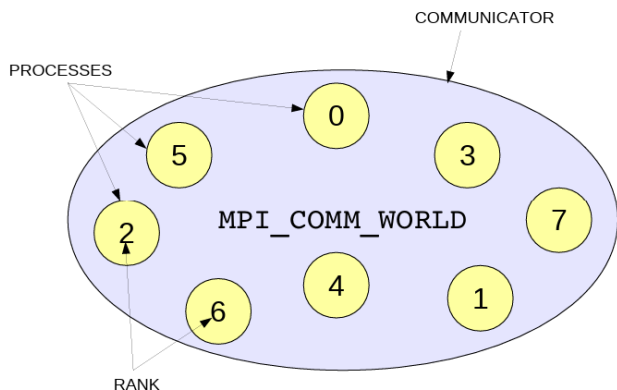
La gestione dei processi sui diversi nodi avviene tramite demoni (Orted in openMPI) che comunicano tra loro via ssh (o altri protocolli di rete).

mpirun realizza il modello SPMD (Single Program Multiple Data): lo stesso programma viene eseguito su diversi processi detti task.

Ogni task ha la propria memoria e i propri dati ed è identificato da un numero intero denominato rank nel range [0, N-1].

Il numero di task e la loro distribuzione sui nodi è definita attraverso le opzioni di mpirun.

Il numero di processi attivabili su un nodo è detto slot.



Le opzioni non sono standardizzate e possono variare tra le diverse implementazioni e versioni. Opzioni principali di mpirun in openMPI

- n <n> : numero di processi da attivare
- npernode <n> : processi per nodo
- host <node list> : lista dei nodi su cui attivare i processi
- hostfile <filename> : file con l'elenco degli host e il numero di slot per host

Esempio di file (hostfile.txt):

```
wn1 slots=4  
wn2 slots=4
```

Esercizi sul PC personale o su login.hpc.unipr.it :

```
module load gnu8 openmpi4 # necessario sul cluster
```

```
mpirun -n 4 date # crea 4 task sull'host locale in cui viene eseguito date.
```

Esercizi che coinvolgono i Worker Nodes (wn):

```
mpirun -host wn1:4,wn2:2 -n 5 hostname # sul PC
```

```
mpirun -host wn21:4,wn22:4 -npernode 2 hostname # sul cluster
```

```
mpirun -hostfile hostfile.txt -n 2 hostname
```

```
mpirun -hostfile hostfile.txt -npernode 4 hostname
```

mpicc

MPI fornisce una libreria per lo scambio di messaggi tra i processi.

Per utilizzare le primitive della libreria si utilizza un compilatore specifico che è un wrapper del compilatore di base.

Linguaggio	Default Compiler	MPI wrapper
Gnu C	gcc	mpicc
Gnu C++	g++	mpicxx o mpic++
fortran	gfortran	mpifc
Intel C	icc	mpiicc o mpicc
Intel C++	icpc	mpiicpc

Compilazione ed esecuzione dei programmi di base

Il file `mpi_base.tgz` disponibile nella consegna e nella directory `SHARE/HPC2526/` contiene alcuni esercizi dimostrativi dei principali schemi di comunicazione MPI.

Analizzare le primitive di comunicazione contenute negli esercizi in prospettiva di un loro possibile utilizzo nelle applicazioni `cpi`, `heat` e `factorize`.

Compilare gli esercizi con `mpicc` ed eseguirli con `mpirun`. Esempio:

```
module load gnu8 openmpi4 # in caso di cluster HPC
mpicc mpi1_helloworld.c -o mpi1_helloworld
mpirun mpi1_helloworld
mpirun -np 16 -host localhost mpi1_helloworld
```

Esecuzione via SLURM

Grazie all'integrazione tra Slurm e `mpirun`, le risorse selezionate via Slurm vengono automaticamente comunicate a `mpirun`,

Principali opzioni Slurm per la selezione delle CPU:

https://www.hpc.unipr.it/dokuwiki/doku.php?id=calcoloscientifico:userguide#main_slurm_options

Corrispondenza tra direttive Slurm e opzioni di mpirun in open-mpi:

	slurm	mpirun
Numero di task	-n --ntasks	-n
Numero di nodi	-N --nodes	elenco in -host
Task per nodo	--ntasks-per-node	-npernode
Cpu per task (*)	--cpus-per-task	

(*) numero di core riservati per ogni task; definisce il numero eventuale di thread openMP da attivare per processo.

L'elenco delle risorse assegnate da Slurm può essere visualizzato consultando le variabili di ambiente.

Le variabili utili per l'esercizio sono:

\$SLURM_NTASKS (numero di task attivati)

\$SLURM_JOB_NODELIST (elenco dei nodi assegnati da Slurm)

\$SLURM_JOB_CPUS_PER_NODE (numero di cpu assegnate per nodo)

Analizzare ed eseguire lo script Slurm

Consegna: caricare il file di output dell'esecuzione degli script slurm riportati nella consegna.